Belief Revision Engine

DTU 02180 – Introduction to Artificial Intelligence
Group Assignment 2, Spring 2026

Group AI3

April 2026

Contents

1 Introduction

A central challenge in knowledge representation is how to update an agent's beliefs in the light of new information. As posed in the course slides, *"how do you update a database of knowledge in the light of new information?"* When an agent acquires a new observation φ that contradicts some of its existing beliefs, it cannot simply add φ to its belief set without first removing the conflicting material, because an inconsistent belief set entails everything and is therefore useless.

We address this problem using the *belief base approach*: beliefs are represented as an ordered, finite set of propositional formulas equipped with a *priority ordering* that encodes degrees of entrenchment. We implement three belief-change operations:

- **Expansion** $B + \varphi$: add new information unconditionally.
- **Contraction** $B \div \varphi$: remove a belief and all formulas that jointly entail it, while preserving the most entrenched beliefs.
- **Revision** $B * \varphi$: incorporate potentially conflicting new information, defined via the *Levi Identity*: $B * \varphi := (B \div \neg\varphi) + \varphi$.

Logical entailment is decided by the resolution algorithm of Russell & Norvig (2020, henceforth AIMA), implemented from scratch without any external SAT solver. Contraction uses *partial meet contraction* with a priority-based selection function. We verify the implementation against the five AGM revision postulates.

2 Belief Base

2.1 Propositional language

We work over the propositional language $\mathcal{L}$ generated by a countable set of *atoms* $\{p, q, r, \ldots\}$ and five connectives $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$. The set of well-formed formulas (WFF) is defined by the grammar

$$\varphi \ ::= \ p \ \mid \ \neg\varphi \ \mid \ (\varphi \wedge \varphi) \ \mid \ (\varphi \vee \varphi) \ \mid \ (\varphi \rightarrow \varphi) \ \mid \ (\varphi \leftrightarrow \varphi)$$

following AIMA Figure 7.7. Atoms are the atomic sentences; a *literal* is an atom p or its negation $\neg p$.

2.2 Belief base with priority ordering

Following the belief-base approach (as opposed to the belief-set approach based on plausibility orders, also discussed in the slides), we represent the agent's epistemic state as a finite ordered list

$$B = [\varphi_0, \varphi_1, \ldots, \varphi_{n-1}]$$

where the *index* of a formula encodes its *entrenchment*: φ_0 is the most entrenched belief (it will be the last to be removed during contraction) and φ_{n-1} is the least entrenched.

Example belief base (from slides9, "Bob"):

$$B = [p, \ q, \ (p \rightarrow q)] \quad \text{with priority } p > q > (p \rightarrow q).$$

This represents the state where Bob believes p with highest confidence, q with medium confidence, and $p \rightarrow q$ with the lowest confidence among the three.

3 Logical Entailment via Resolution

3.1 Entailment

We write $KB \models \alpha$ to mean that α is a *logical consequence* of KB: in every model (truth-value assignment) in which all formulas in KB are true, α is also true (AIMA §7.3). The logical closure of B is $\mathrm{Cn}(B) = \{\varphi \mid B \models \varphi\}$.

3.2 CNF conversion

The resolution procedure requires the input to be in *conjunctive normal form* (CNF): a conjunction of clauses, where each clause is a disjunction of literals (AIMA §7.5.2). We convert any formula to CNF in four steps (AIMA Figure 7.12):

1. **Eliminate biconditionals**: replace $(\alpha \leftrightarrow \beta)$ with $(\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$.
2. **Eliminate implications**: replace $(\alpha \rightarrow \beta)$ with $(\neg\alpha \vee \beta)$.
3. **Move negations inward**: apply double-negation elimination $(\neg\neg\alpha \equiv \alpha)$ and De Morgan's laws: $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$, $\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$.
4. **Distribute $\vee$ over $\wedge$**: $(\alpha \vee (\beta \wedge \gamma)) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$.

Worked example: convert $B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$ (from AIMA §7.5.2):

$$\begin{aligned}
&\text{Step 1:} && (B_{1,1} \rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \rightarrow B_{1,1}) \\
&\text{Step 2:} && (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1}) \\
&\text{Step 3:} && (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1}) \\
&\text{Step 4:} && (\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})
\end{aligned}$$

The result is a conjunction of three clauses, each a disjunction of literals.

3.3 Resolution rule and proof by refutation

The **resolution rule** (AIMA §7.5.2) states: from a clause $(\ell \vee C_1)$ and a clause $(\neg\ell \vee C_2)$, we may derive the **resolvent** $C_1 \vee C_2$, where ℓ and $\neg\ell$ are complementary literals.

To decide $KB \models \alpha$, we use **proof by refutation** (AIMA Fig. 7.13): convert $(KB \wedge \neg\alpha)$ to CNF, then repeatedly apply the resolution rule to pairs of clauses, adding each new resolvent to the clause set. If the *empty clause* (denoted $\square$, representing $\bot$) is derived, then $KB \wedge \neg\alpha$ is unsatisfiable, so $KB \models \alpha$. If no new clauses can be added (fixpoint reached), then $KB \not\models \alpha$.

The algorithm is **sound**: every resolvent is a logical consequence of its parent clauses. It is **complete** by the *Ground Resolution Theorem* (AIMA §7.5.2): if a set of clauses is unsatisfiable then its resolution closure contains the empty clause.

Small worked example: check $\{p,\ p \rightarrow q\} \models q$:

1. Add $\neg q$ (negated query). CNF clauses: $\{p\}$, $\{\neg p, q\}$, $\{\neg q\}$.
2. Resolve $\{p\}$ with $\{\neg p, q\}$: get $\{q\}$.
3. Resolve $\{q\}$ with $\{\neg q\}$: get $\square$ (empty clause).
4. Empty clause derived $\Rightarrow \{p, p \rightarrow q\} \models q$. ✓

4 Contraction

4.1 Remainder sets

For a belief base B and formula φ, the **remainder set** $B \perp \varphi$ is the collection of all *maximal* subsets of B that do not entail φ:

$$B \perp \varphi \;=\; \big\{S \subseteq B \;\mid\; S \not\models \varphi \;\text{ and }\; \forall\, S' \text{ s.t. } S \subsetneq S' \subseteq B : S' \models \varphi\big\}.$$

The *maximality* condition ensures we remove as little as possible.

4.2 Selection function and partial meet contraction

We use a priority-based **selection function** γ: for each remainder $S \in B \perp \varphi$, compute the sorted tuple of priority indices of its members. We select all remainders whose sorted index tuple is lexicographically minimal (equivalently, that retain the most-entrenched beliefs).

Partial meet contraction is defined as:

$$B \div \varphi \;:=\; \bigcap \gamma(B \perp \varphi).$$

That is, $B \div \varphi$ contains exactly the formulas present in *every* selected remainder.

Worked example: $B = [p,\ q,\ (p \to q)]$, contract by q.

Since $B \models q$, contraction is non-vacuous. The remainders $B \perp q$ are:

- $S_1 = \{p\}$ (removing q and $(p \to q)$ yields a set not entailing q)
- $S_2 = \{(p \to q)\}$ (similarly)
- $S_3 = \{p, q \text{ removed}, (p \to q) \text{ removed}\} = \{p\}$ — same as S_1

More precisely, the maximal subsets of B not entailing q are: $\{p\}$ and $\{(p \to q)\}$. The selection function prefers $\{p\}$ (contains index-0 element p, the most entrenched). Thus $\gamma(B \perp q) = \{\{p\}\}$ and $B \div q = \{p\}$.

4.3 Contraction postulates (informal)

Our implementation satisfies the following AGM contraction postulates:

Failure If $B \not\models \varphi$, then $B \div \varphi = B$ (vacuous contraction): implemented by an immediate check.

Success $B \div \varphi \not\models \varphi$ (unless φ is a tautology): guaranteed because we only keep subsets of B that do not entail φ.

Inclusion $B \div \varphi \subseteq B$: the contraction is a subset of the original base by construction.

Relevance Every formula dropped in going from B to $B \div \varphi$ was relevant to entailing φ: guaranteed by the maximality of each remainder.

Uniformity If any subset of B that implies φ also implies ψ, then $B \div \varphi$ and $B \div \psi$ keep the same formulas: satisfied since selection depends only on the structure of remainders, which is symmetric with respect to φ and ψ under this condition.

5 Expansion

Expansion is the simplest belief-change operation. For a formula φ:

$$B + \varphi \;:=\; \mathrm{Cn}(B \cup \{\varphi\}).$$

In a finite syntactic implementation, we simply add φ to the list of formulas (at the lowest priority, i.e. appended at the end). Expansion always succeeds in adding φ to the belief base, but

it does not check for consistency: expanding with $\neg q$ when $q \in B$ produces an inconsistent belief base. This is why revision (which first removes the obstacle) is the more appropriate operation when the incoming information may conflict with existing beliefs.

6 Revision

6.1 Levi Identity

Revision is defined via the **Levi Identity** (slides9):

$$B * \varphi \;:=\; (B \div \neg\varphi) + \varphi.$$

The rationale is to first remove from B all formulas that would block the acceptance of φ (contraction by $\neg\varphi$), and then add φ (expansion). This two-step process ensures that φ is accepted and that the result is consistent whenever φ itself is consistent.

6.2 AGM revision postulates

We verify the five AGM revision postulates (slides9). All 22 tests in our test suite pass:

(R1) Success $\varphi \in \mathrm{Cn}(B * \varphi)$. *Satisfied*: after revision, φ is in the expanded base by definition of expansion.

(R2) Inclusion $B * \varphi \subseteq B + \varphi$. *Satisfied*: the contracted base $(B \div \neg\varphi)$ is a subset of B, so $(B \div \neg\varphi) + \varphi \subseteq B + \varphi$.

(R3) Vacuity If $\neg\varphi \notin \mathrm{Cn}(B)$, then $B * \varphi = B + \varphi$. *Satisfied*: if $B \not\models \neg\varphi$, then $B \div \neg\varphi = B$ (Failure postulate of contraction), so $B * \varphi = B + \varphi$.

(R4) Consistency $B * \varphi$ is consistent unless φ is a contradiction. *Satisfied*: after contracting $\neg\varphi$, the base does not entail $\neg\varphi$. Adding φ then yields a consistent base (since φ is consistent and does not conflict with the contracted base).

(R5) Extensionality If $\varphi \equiv \psi$ then $B * \varphi = B * \psi$. *Satisfied*: logically equivalent formulas have the same negation (modulo equivalence), so $B \div \neg\varphi$ and $B \div \neg\psi$ yield equivalent bases, and adding equivalent φ or ψ preserves equivalence.

6.3 Worked example

Let $B = [p,\ q,\ (p \to q)]$ and revise by $\neg q$:

1. **Contraction** $B \div q$ (since $\neg(\neg q) = q$): Remainders of $B \perp q$ are $\{p\}$ and $\{(p \to q)\}$. Selection function chooses $\{p\}$ (most entrenched). $B \div q = \{p\}$.
2. **Expansion** $\{p\} + \neg q = \{p,\ \neg q\}$.
3. **Result**: $B * \neg q = \{p,\ \neg q\}$.

Checking the postulates: (R1) $\neg q \in B * \neg q$ ✓; (R4) $\{p, \neg q\}$ is consistent ✓; (R3) not applicable since $B \models q = \neg(\neg q)$ ✓.

7 What We Learned

Implementing the belief revision engine from scratch taught us several lessons.

CNF and resolution. Implementing full CNF conversion correctly required careful attention to the distribution of $\vee$ over $\wedge$. Formulas such as $(p \vee (q \wedge r)) \vee (s \wedge t)$ require multiple recursive passes. We also had to handle tautological clauses (those containing both p and $\neg p$) separately, since they would otherwise cause spurious entailments.

Remainder-set enumeration. Finding all maximal non-entailing subsets is exponential in the size of the belief base (there are up to 2^n subsets for a base of size n). For small bases this is manageable, but for larger belief bases a more efficient representation — e.g., a partial order over formulas — would be necessary. We also found it subtle to ensure *maximality*: a naive approach that removes one formula at a time does not always produce all maximal subsets.

Theory vs. implementation. The AGM framework is defined at the level of logical consequences ($\mathrm{Cn}(B)$), while our implementation works at the syntactic level (finite lists of formulas). Some care is needed when comparing belief bases for semantic equality (we check mutual entailment rather than syntactic identity). The Levi Identity elegantly bridges contraction and revision, and its correctness is easy to verify at the semantic level. Translating this to the syntactic level required showing that the priority-based selection function tracks the intended semantic notion of "most plausible world."

8 Conclusion and Further Work

We have implemented a complete, working belief revision engine in Python using only the standard library. The engine supports:

- Full propositional formula parsing with all five connectives.
- CNF conversion following the four-step algorithm of AIMA Chapter 7.
- A resolution-based entailment checker (PL-Resolution, AIMA Fig. 7.13).
- Partial meet contraction with a priority-based selection function.
- Expansion and revision via the Levi Identity.
- An AGM postulate test suite (22 tests, all passing).

Several directions for further work suggest themselves:

Efficiency The remainder-set enumeration is exponential. Clever indexing of clauses (which literals appear positive/negative in which clauses) and efficient data structures for the clause set would substantially speed up the resolution procedure.

Iterated revision The current implementation performs single-step revision. Iterated revision (applying revision repeatedly as new information arrives) requires a dynamic representation of entrenchment, e.g., an epistemic entrenchment ordering or a TPO (total pre-order over possible worlds) as discussed in the slides.

Plausibility-order approach The slides present an alternative formulation of belief revision in terms of plausibility orders over valuations. It would be interesting to implement this approach and compare it with the belief-base approach for the same examples.

Mastermind extension The assignment mentions an optional Mastermind extension. The belief base could model the game's constraints as propositional formulas, and revision could be used to narrow down possible solutions after each guess.

References

[1] Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4th ed. Chapter 7: Logical Agents.

[2] Gärdenfors, P. (1988). *Knowledge in Flux: Modelling the Dynamics of Epistemic States.* MIT Press. (As presented in DTU 02180 slides9.)

[3] DTU 02180 Lecture Slides, Slides 8 and 9: Propositional Logic, Resolution, Belief Revision (Spring 2026).